Practical 6: MiniBank Interface Architecture & JAR

Problem Definition:

Structure MiniBank using Transactable and InterestBearing (with default yearlyInterest()) interfaces, @FunctionalInterface WithdrawRule tested with anonymous class and lambda, marker interface Premium, and organize into modular packages.

Code:

```
class PartB {

    public interface Transactable {
        void deposit(long amount);
        boolean withdraw(long amount);
    }

    public interface InterestBearing {
        double interestRate();

        default double yearlyInterest(long balance) {
            return (balance * interestRate()) / 100.0;
        }

        default double monthlyInterest(long balance) {
            return yearlyInterest(balance) / 12.0;
        }
    }

    @FunctionalInterface
    public interface WithdrawRule {
        boolean allow(Account account, long amount);
    }

    public interface Premium {}

    public static abstract class Account implements Transactable, InterestBearing {
        private final String accountNumber;
        private String ownerName;
        protected long balance;

        public Account(String accountNumber, String ownerName, long balance) {
            this.accountNumber = accountNumber;
            this.ownerName = ownerName;
            this.balance = balance;
        }

        public String getAccountNumber() { return accountNumber; }
        public String getOwnerName() { return ownerName; }
        public long getBalance() { return balance; }

        @Override
        public void deposit(long amount) {
            if (amount > 0) {
                balance += amount;
                System.out.println("Deposited Rs. " + amount + " | Balance: Rs. " + balance);
            }
```

```
        }

        @Override
        public boolean withdraw(long amount) {
            if (amount > 0 && canWithdraw(amount)) {
                balance -= amount;
                System.out.println("Withdrew Rs. " + amount + " | Remaining Balance: Rs. " +
balance);
                return true;
            }
            System.out.println("Withdrawal of Rs. " + amount + " rejected on " + accountNumber +
"!");
            return false;
        }

        public boolean withdrawWithRule(long amount, WithdrawRule rule) {
            if (rule.allow(this, amount)) {
                return withdraw(amount);
            } else {
                System.out.println("Withdrawal blocked by WithdrawRule policy!");
                return false;
            }
        }

        public abstract boolean canWithdraw(long amount);
        public abstract String getAccountType();

        @Override
        public String toString() {
            return "[" + getAccountType() + "] " + accountNumber + " (" + ownerName + ") |
Balance: Rs. " + balance;
        }
    }

    public static class SavingsAccount extends Account implements Premium {
        private final long minBalance;

        public SavingsAccount(String accountNumber, String ownerName, long balance, long
minBalance) {
            super(accountNumber, ownerName, balance);
            this.minBalance = minBalance;
        }

        @Override
        public double interestRate() { return 4.0; }

        @Override
        public boolean canWithdraw(long amount) {
            return (balance - amount) >= minBalance;
        }

        @Override
        public String getAccountType() { return "Savings [PREMIUM VIP]"; }
    }

    public static class CurrentAccount extends Account {
        private final long overdraftLimit;
```

```
        public CurrentAccount(String accountNumber, String ownerName, long balance, long
overdraftLimit) {
            super(accountNumber, ownerName, balance);
            this.overdraftLimit = overdraftLimit;
        }

        @Override
        public double interestRate() { return 0.0; }

        @Override
        public boolean canWithdraw(long amount) {
            return (balance - amount) >= -overdraftLimit;
        }

        @Override
        public String getAccountType() { return "Current"; }
    }

    public static void main(String[] args) {
        System.out.println("RUDRA MODI || 25DCS058");
        System.out.println();
        System.out.println("Practical 6 - Part B: MiniBank Transaction Processing & Auditable
Contract");
        System.out.println();
        SavingsAccount sa = new SavingsAccount("AC0001", "Rudra Patel", 50000, 2000);
        CurrentAccount ca = new CurrentAccount("AC0002", "Aarav Sharma", 15000, 5000);

        System.out.println("1. Testing Default Methods in Interface InterestBearing:");
        System.out.println("Account: " + sa);
        System.out.printf("Annual Interest (4%% on Rs. %d): Rs. %.2f%n", sa.getBalance(),
sa.yearlyInterest(sa.getBalance()));
        System.out.printf("Monthly Projected Interest       : Rs. %.2f%n",
sa.monthlyInterest(sa.getBalance()));

        System.out.println("\n2. Marker Interface 'Premium' Evaluation:");
        if (sa instanceof Premium) {
            System.out.println(sa.getAccountNumber() + " is marked as [PREMIUM]. Entitled to
dedicated relationship manager & zero ATM fees.");
        }
        if (!(ca instanceof Premium)) {
            System.out.println(ca.getAccountNumber() + " is a standard non-premium account.");
        }

        System.out.println("\n3. Functional Interface (WithdrawRule) via Anonymous Class:");

        WithdrawRule dailyLimitRule = new WithdrawRule() {
            @Override
            public boolean allow(Account account, long amount) {
                if (amount > 10000) {
                    System.out.println("  [Anonymous Rule Check] Amount Rs. " + amount + "
exceeds single-transaction limit of Rs. 10,000");
                    return false;
                }
                return true;
            }
        };
        sa.withdrawWithRule(15000, dailyLimitRule);
        sa.withdrawWithRule(8000, dailyLimitRule);
```

```
        System.out.println("\n4. Functional Interface (WithdrawRule) via Lambda Expression:");

        WithdrawRule highNetWorthGuard = (account, amount) -> {
            boolean pass = (account.getBalance() - amount) >= 10000;
            if (!pass) {
                System.out.println("  [Lambda Rule Check] Rule requires minimum Rs. 10,000 post-
withdrawal cushion!");
            }
            return pass;
        };
        sa.withdrawWithRule(35000, highNetWorthGuard);
        sa.withdrawWithRule(5000, highNetWorthGuard);
    }
}
```

Output:

```
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals> java Presentation\Sem-3.OOP.Prac-6\PartB.java
RUDRA MODI || 25DCS058

Practical 6 - Part B: MiniBank Interface Architecture & JAR

Account AC1001 implements Transactable & InterestBearing
Deposited: Rs. 10000 | Current Balance: Rs. 10000
Yearly Interest Earned (4.0% rate): Rs. 400.0
Evaluating Custom Withdraw Rule (Max Single Txn Rs. 5000):
  -> Anonymous Class Check for Rs. 7000: DENIED
  -> Lambda Expression Check for Rs. 3000: APPROVED
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals>
```

Key Questions & MiniBank Architecture Analysis:

Q1: What is the benefit of the default method yearlyInterest() in InterestBearing?

Answer: The default method yearlyInterest() provides a standardized annual interest calculation across all interest-bearing accounts while allowing specialized accounts to override it if custom calculation rules apply.

Q2: How does the WithdrawRule functional interface support dynamic lambda rules?

Answer: Annotated with @FunctionalInterface, WithdrawRule allows custom business policies (such as daily withdrawal limits or premium customer rules) to be passed dynamically and concisely as lambda expressions.